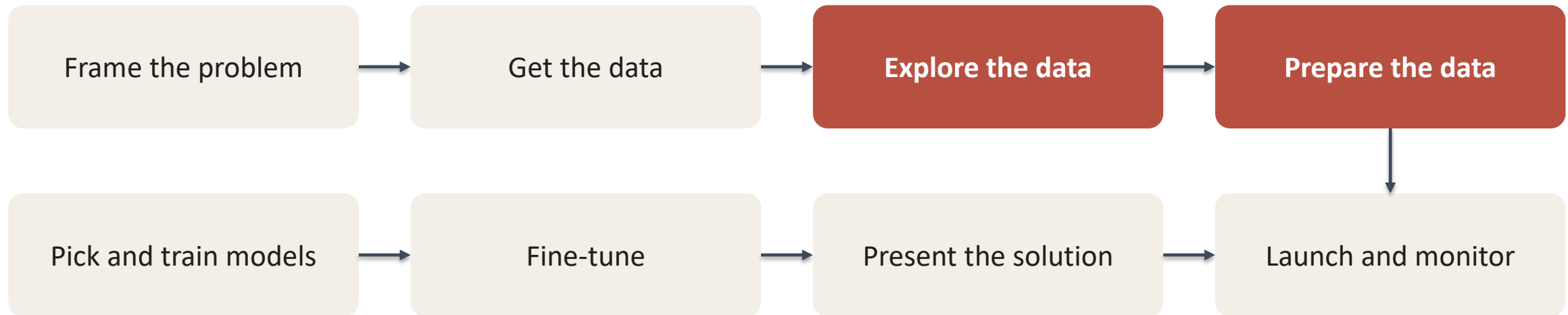


Data exploration and preprocessing

Most of the work in a machine learning project is not the model. A rule of thumb: about 80 percent of the time goes to getting the data ready.



Today: the two steps in red. Week 3 onward is the rest of the bottom row.

Explore means looking before touching: distributions, maps, correlations, oddities.

Prepare means the transformations a model needs: missing values, scaling, encoding, and a clean split into training, validation, and test sets.

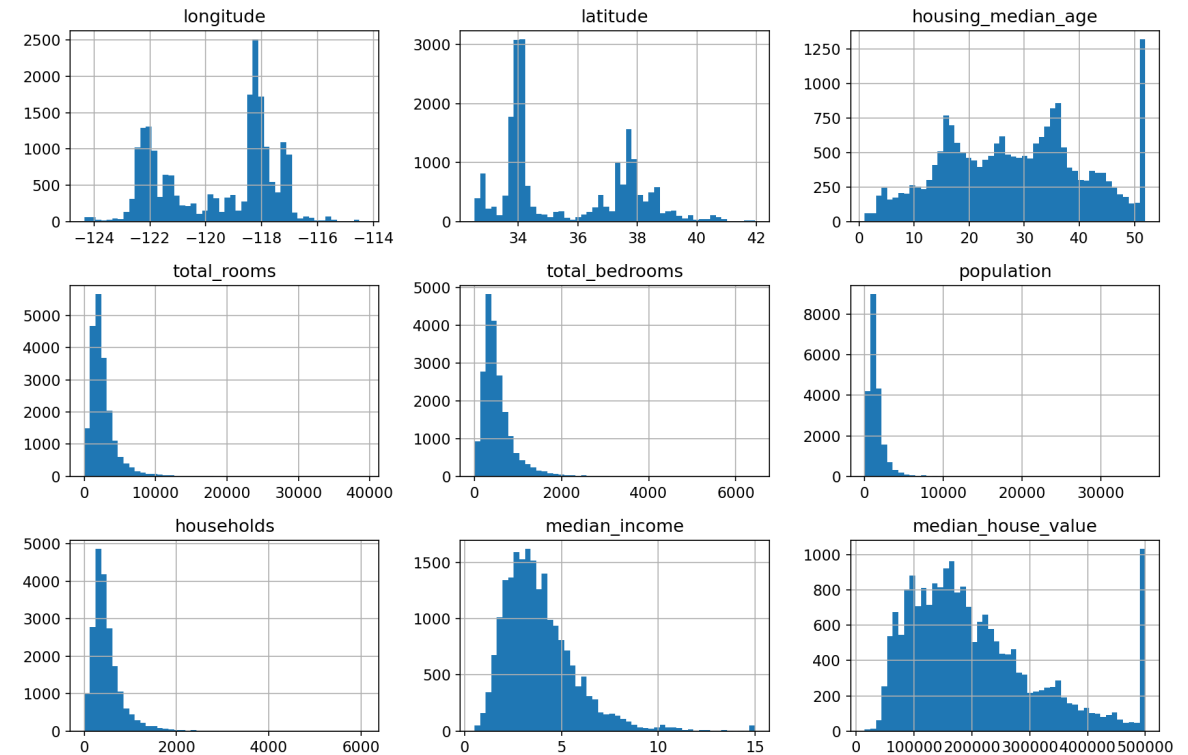
Look at the data before you model it

- **Géron's running example:** California housing, one row per census district, 20,640 rows and 10 columns

- **First move: a histogram of every column.**

What to look for:

- Units and scale. Median income is in tens of thousands of dollars, and is capped at 15
- Capped values. House value stops at \$500,000, house age at 52. The model will never predict above the cap
- Heavy tails. Rooms, population, and households have a few enormous districts
- Very different ranges across columns. That is why scaling comes later



Figures regenerated from the author's notebook for Géron, Hands-On Machine Learning, 3rd ed., Ch. 2 (Apache 2.0 code; California housing data)

The same habit on our data: 55,367 US wildfires

- **fire_size**: almost every fire is tiny and a few are enormous. A log transform is coming
- **Temp, Wind, Hum, Prec**: spikes at exactly 0 and -1. Those are missing values written as numbers, not measurements
- **Vegetation**: bars at integer codes. It is a category, not a quantity, even though it is stored as a number
- **latitude**: clusters near 18 and above 60 are Puerto Rico, Hawaii, and Alaska, not errors
- A histogram grid takes one line and catches most of the surprises before they reach the model

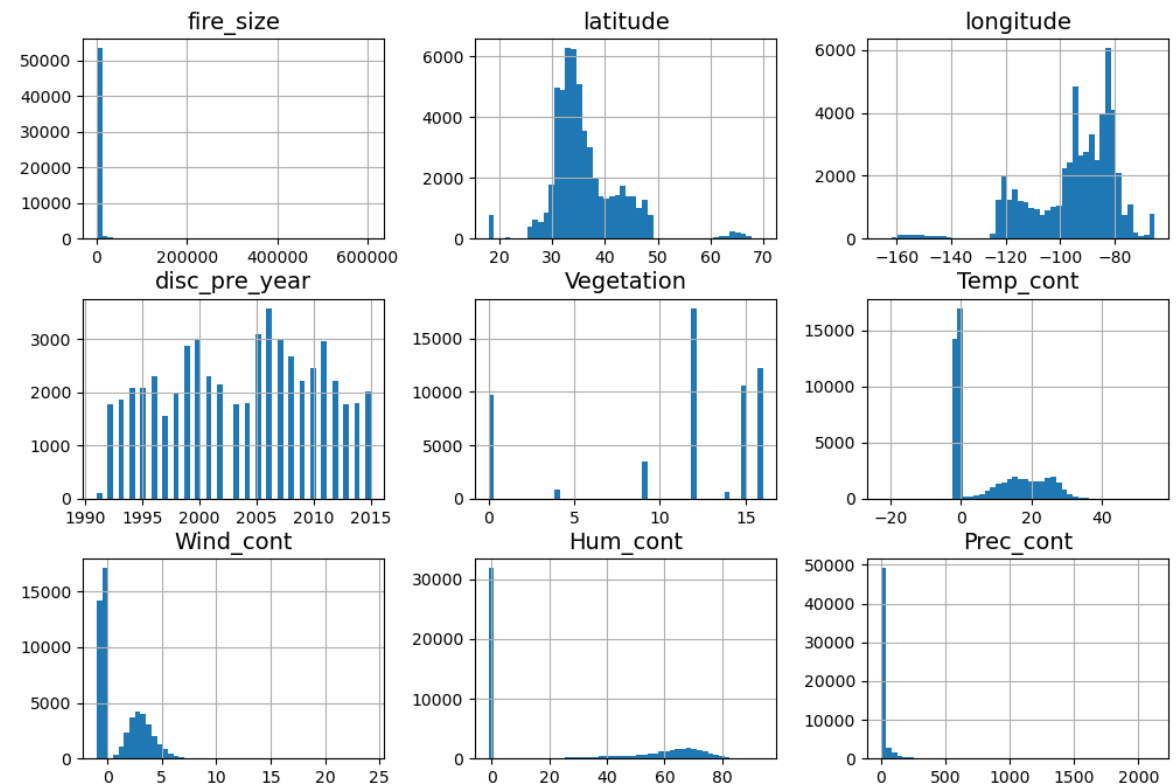
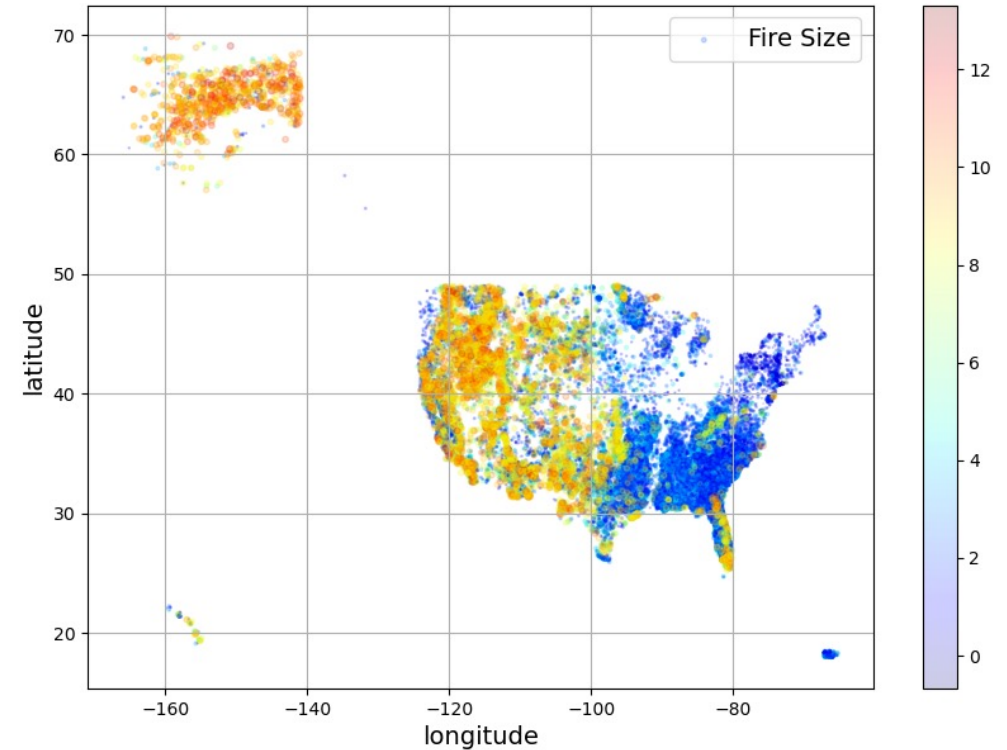
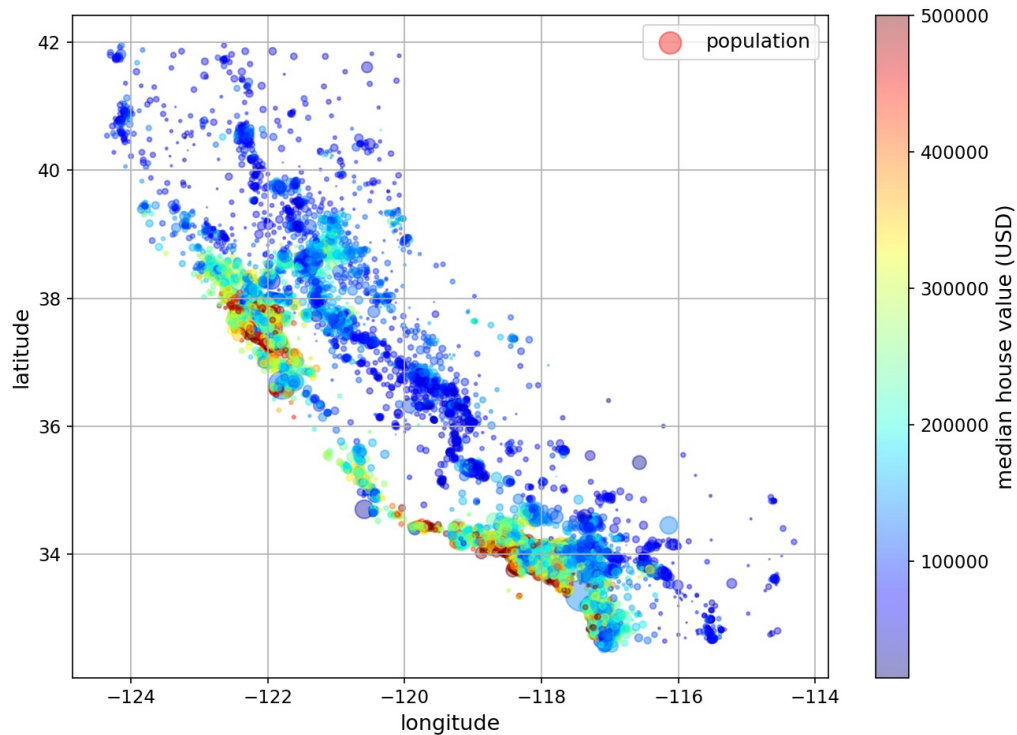


Figure from the Week 2 tutorial notebook (US wildfire data set on Kaggle, derived from the USFS Fire Program Analysis fire-occurrence database)

Geography is a feature: plot it

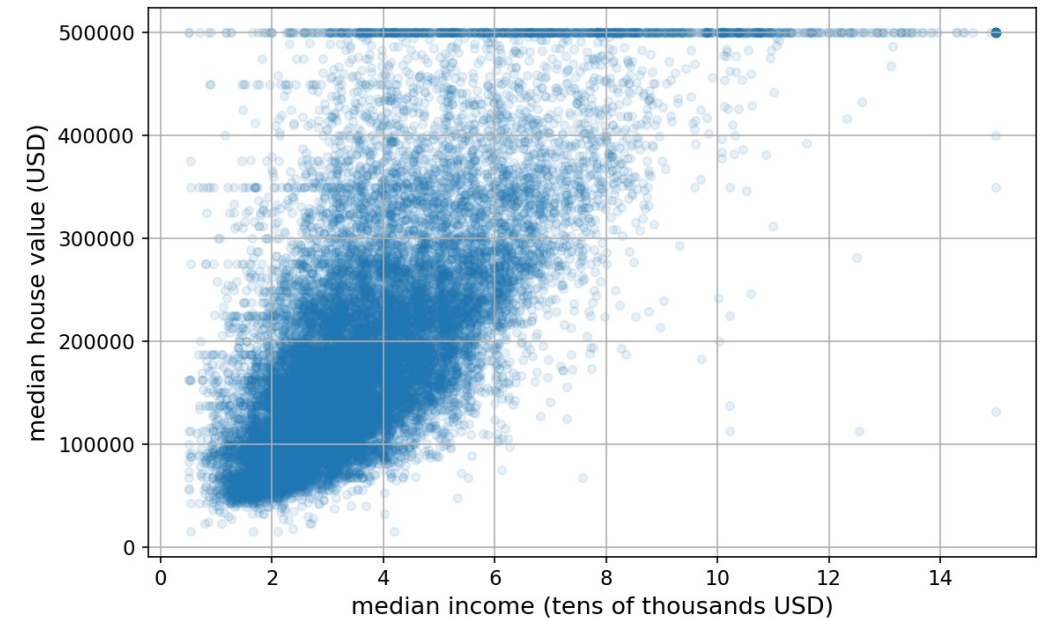
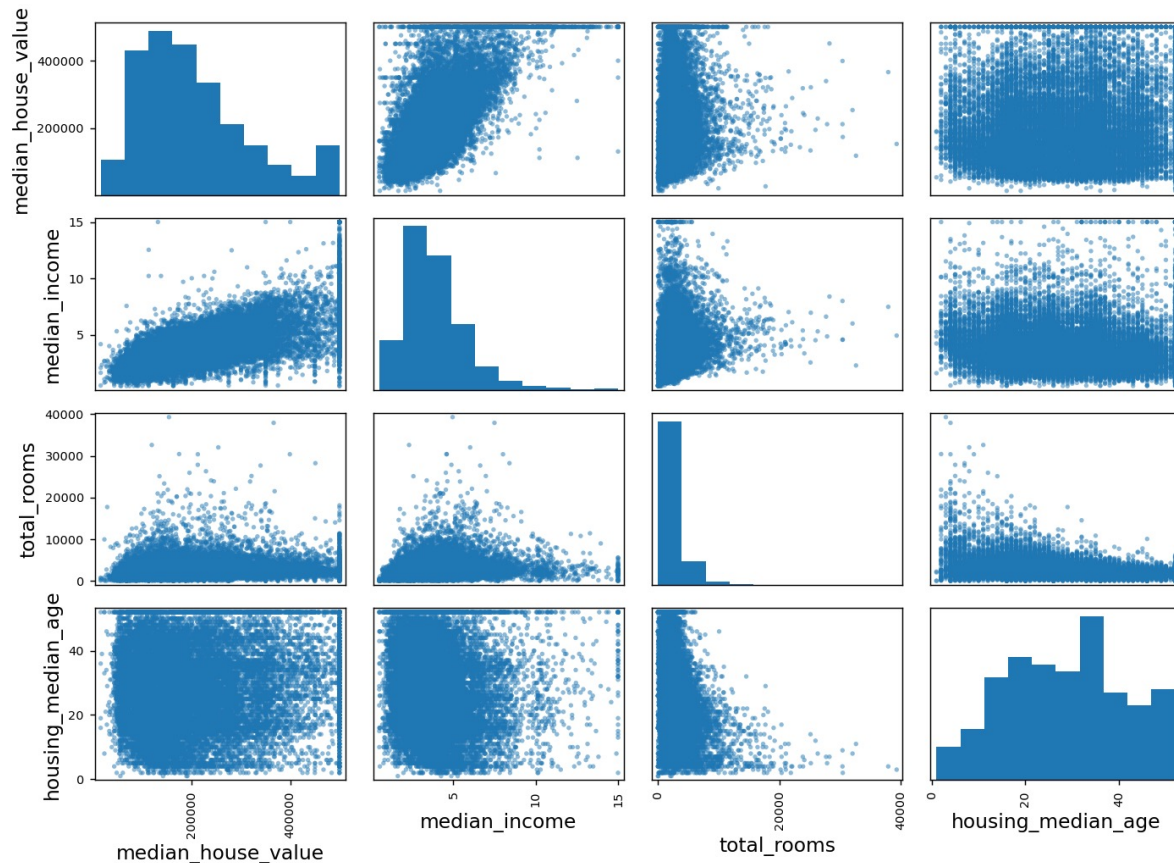
Left: California districts, marker size is population, colour is median house value. Prices track the coast and the big cities. **Right:** fire size across the US. The largest fires are in the West and Alaska.



A pattern that jumps out of a map usually becomes a feature: distance to the ocean, distance to a city, region, elevation.

*Left: Figures regenerated from the author's notebook for Géron, Hands-On Machine Learning, 3rd ed., Ch. 2 (Apache 2.0 code; California housing data).
Right: figure from the Week 2 tutorial notebook (US wildfire data set on Kaggle, derived from the USFS Fire Program Analysis fire-occurrence database)*

Scatter plots find relationships, and quirks



Income is the strongest predictor of price (correlation 0.69), and the relationship is close to linear.

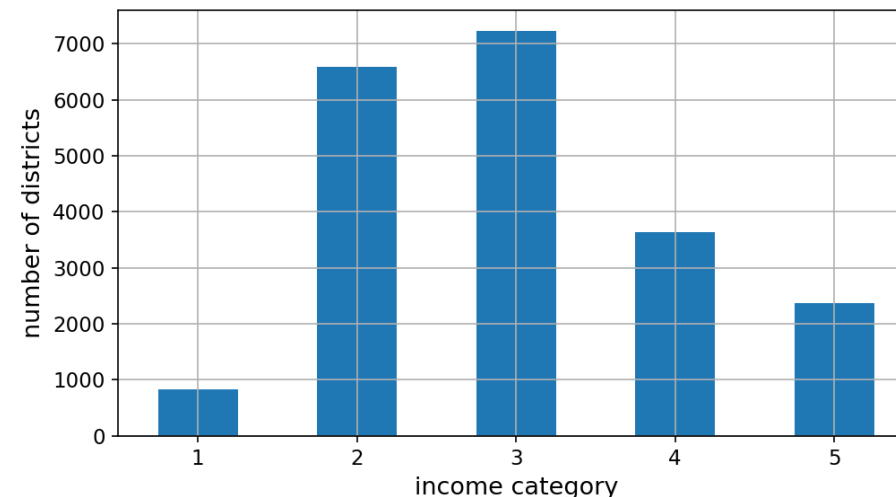
The horizontal lines at \$500,000, and fainter ones near \$450,000 and \$350,000, are survey artefacts. A model will learn to reproduce them. Remove those districts, or accept the ceiling.

`pandas.plotting.scatter_matrix` draws every pair of numeric columns. Diagonal panels are histograms.

Figures regenerated from the author's notebook for Géron, Hands-On Machine Learning, 3rd ed., Ch. 2 (Apache 2.0 code; California housing data)

Set the test set aside early

- **Before you study the data too closely, split off a test set and do not look at it.**
 - Your brain is a pattern matcher. Spot a trend in the test set and you will choose a model that fits it. Géron calls this data snooping bias
- **A random split is fine for a large data set.** With a small one, make the split representative: stratify on a variable that matters
- **Géron's example:** bin median income into 5 categories and sample the same share from each. Random sampling gets category 1 wrong by 6 percent, stratified by 0.4 percent



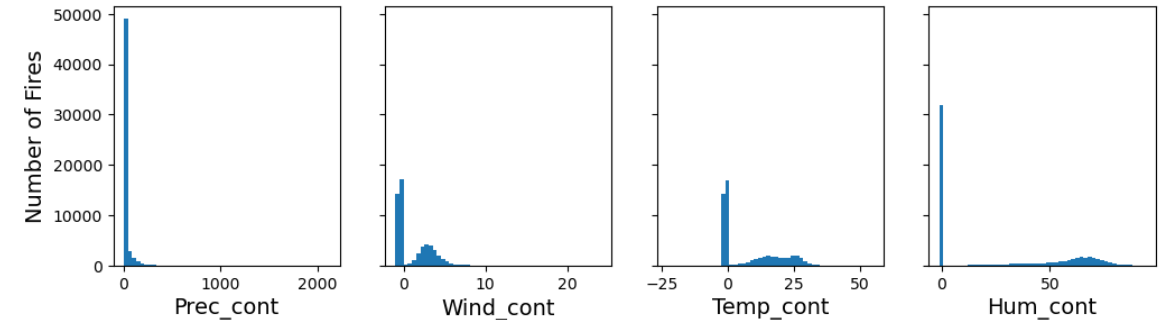
Income category	Overall %	Stratified %	Random %	Random error
1	3.98	4.00	4.24	+6.5%
2	31.88	31.88	30.74	-3.6%
3	35.06	35.05	34.52	-1.5%
4	17.63	17.64	18.41	+4.4%
5	11.44	11.43	12.09	+5.6%

Figures regenerated from the author's notebook for Géron, Hands-On Machine Learning, 3rd ed., Ch. 2 (Apache 2.0 code; California housing data); test set is 20 percent, random_state 42

Missing data: first ask why it is missing

- **putout_time** is missing for half the fires. The fires with no value are much smaller on average. The data are missing for a reason, so dropping those rows changes what the data set is about
- **Missing is not always NaN.** The weather columns use 0 and -1 as codes. Wind of exactly 0.0000 during fire season, or humidity of -1 percent, is a flag, not a measurement
- **Fix the codes first**, `df[col].replace([0.0, -1.0], np.nan)`, then decide what to do with the NaNs
- **Sentinel values are worse than NaN:** pandas will average them in without complaint

Raw columns: the spike at 0 and -1 is the missing data



After replacing 0 and -1 with NaN

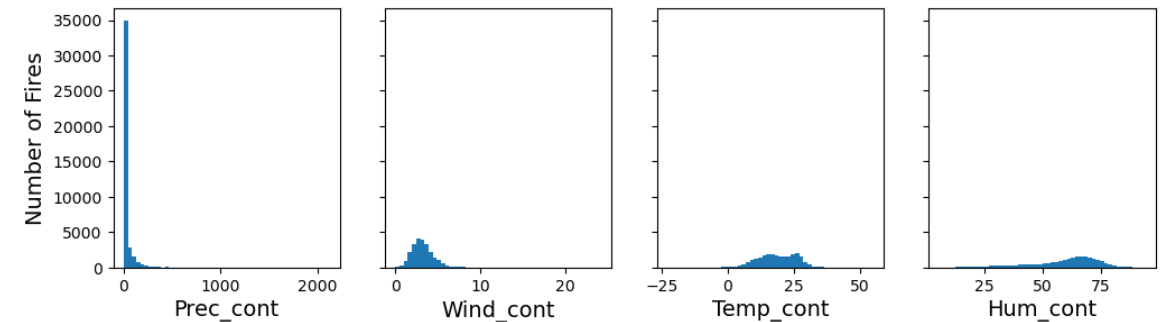
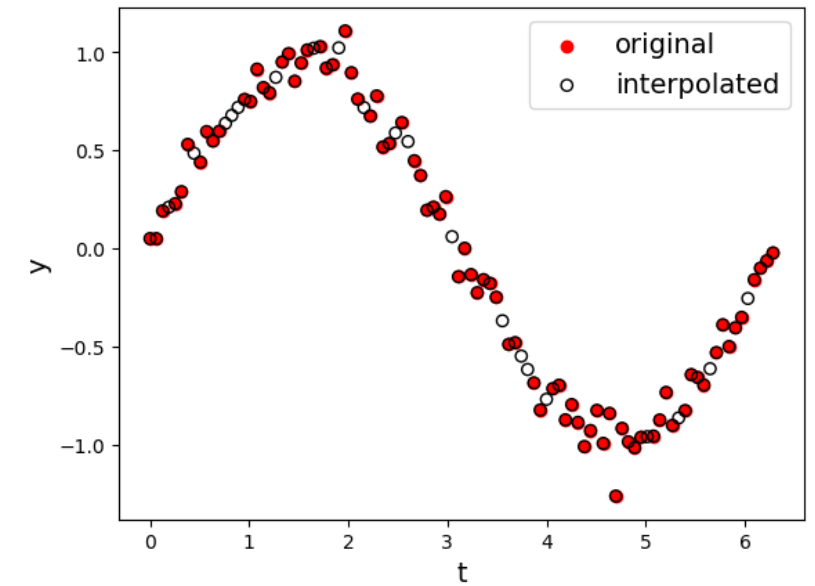


Figure from the Week 2 tutorial notebook (US wildfire data set on Kaggle, derived from the USFS Fire Program Analysis fire-occurrence database)

Four ways to handle the NaNs

Method	Code	Use when	Watch out
Drop the rows	<code>df.dropna()</code>	Plenty of data, values missing at random	The tutorial keeps only 10% of the rows
Fill a constant	<code>df.fillna(0)</code>	The column has a natural default	Invents a measurement that never happened
Mean or median	<code>SimpleImputer(strategy="median")</code>	Small data, numeric columns	Shrinks the variance. Fit on the training set only
Interpolate	<code>df.interpolate()</code>	A time series with gaps	Meaningless for independent records like fires



Interpolation on a sine wave with 20 percent of points removed: sensible here because neighbours in time are related.

Every choice biases the data a little. Choose with the question in mind, record what you did, and check that the training and test sets were treated the same way. Géron's default for numeric columns is the median, learned from the training set inside a pipeline.

*Figure from the Week 2 tutorial notebook (US wildfire data set on Kaggle, derived from the USFS Fire Program Analysis fire-occurrence database).
Methods after Géron, Hands-On Machine Learning, 3rd ed., Ch. 2, "Prepare the data for machine learning algorithms"*

Feature scaling: the scale of the numbers matters

- Fire size runs from 0.1 to 600,000 acres, latitude from 18 to 70, humidity from 0 to 100.
- **Models that measure distances or take gradient steps** treat a difference of 1,000 acres as a thousand times bigger than a difference of 1 degree: k-nearest neighbours, k-means, SVMs, neural networks, regularised regression
- **Tree models do not care.** Decision trees, random forests, and gradient boosting only compare values within a column
- **Scaling changes the numbers, not the shape.** The three histograms below are identical

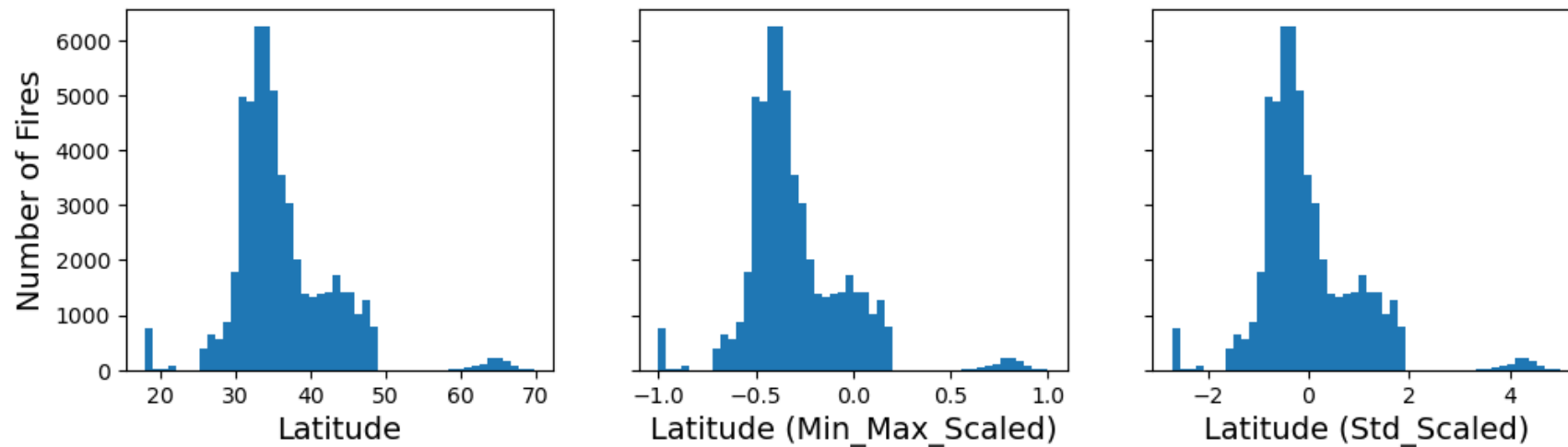


Figure from the Week 2 tutorial notebook (US wildfire data set on Kaggle, derived from the USFS Fire Program Analysis fire-occurrence database): latitude raw, min-max scaled to $(-1, 1)$, and standardised

Two scalers, and when to use which

MinMaxScaler

$$x' = (x - \min) / (\max - \min)$$

Result: every column runs from 0 to 1, or any `feature_range` you set

Use when a bounded range is required: neural network inputs, pixel-like data, or features that are naturally bounded such as humidity

Weakness: one outlier pins the maximum. A single 500,000-acre fire squashes every ordinary fire into the bottom sliver of the range

StandardScaler

$$z = (x - \text{mean}) / \text{std}$$

Result: mean 0 and standard deviation 1, no fixed range. Most values land between -3 and 3

Use when in doubt. The default for linear and logistic regression, SVMs, and most other models

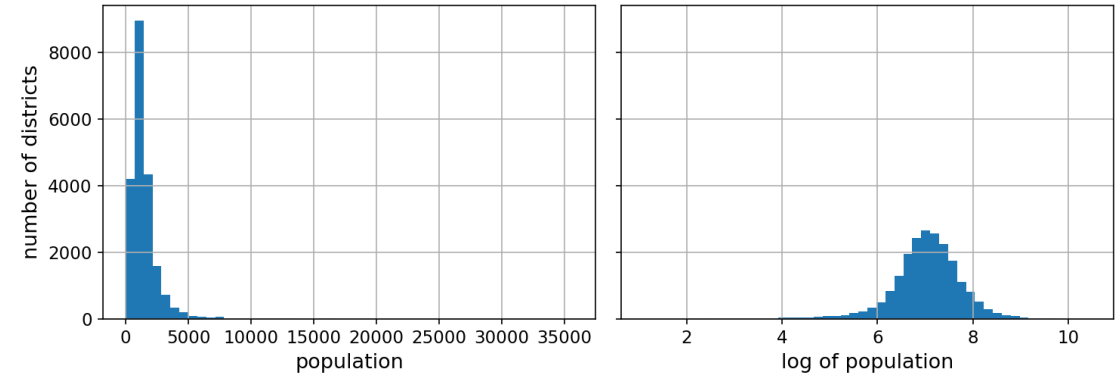
Weakness: none serious. Outliers stay outliers, they just become large z-scores, which is usually what you want

Either way: fit on the training set, then transform everything with those same numbers. Test values can land outside 0 to 1, or beyond ± 3 . That is expected, not an error.

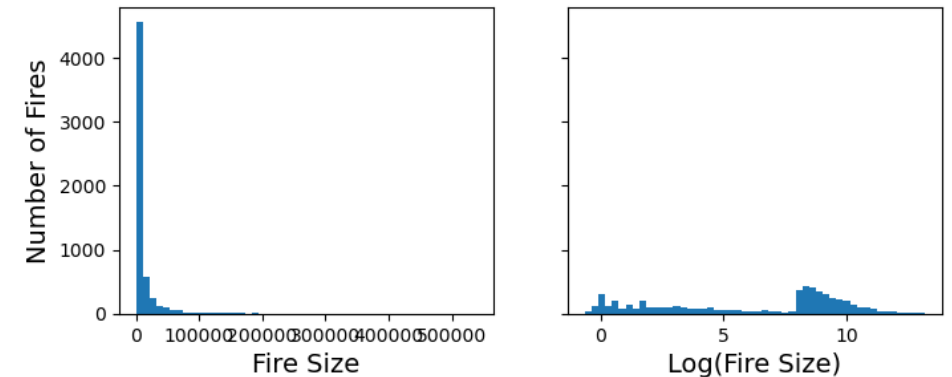
Heavy tails: transform before you scale

- **Population per district and fire size are heavy-tailed:** most values are small, a few are huge
- **Scaling alone does not help.** Min-max squashes the bulk into a sliver, standardising leaves the tail as outliers
- **Take the log first** (or a square root, or a power). The distribution becomes roughly symmetric, then scale as usual
- **In scikit-learn:** `FunctionTransformer(np.log, inverse_func=np.exp)`. The inverse turns predictions back into acres
- **Alternative:** bucketise into quantile bins and treat the bin as a category

California district population



US wildfire size (acres)



Top: Figures regenerated from the author's notebook for Géron, Hands-On Machine Learning, 3rd ed., Ch. 2 (Apache 2.0 code; California housing data).

Bottom: figure from the Week 2 tutorial notebook (US wildfire data set on Kaggle, derived from the USFS Fire Program Analysis fire-occurrence database)

Encoding categorical variables

- **Models want numbers.** `fire_size_class` is a letter A to G, `ocean_proximity` is text, `Vegetation` is a code that only looks numeric
- **Ordinal encoding:** one integer per category. Right when the categories have an order, like fire size class
- **One-hot encoding:** one 0 or 1 column per category. Right when they have no order: state, vegetation, ocean proximity. Otherwise the model believes INLAND sits between ISLAND and NEAR BAY
- **Cost:** one column per category. For hundreds of categories, such as ZIP codes, use embeddings or target encoding instead

OrdinalEncoder on `fire_size_class`

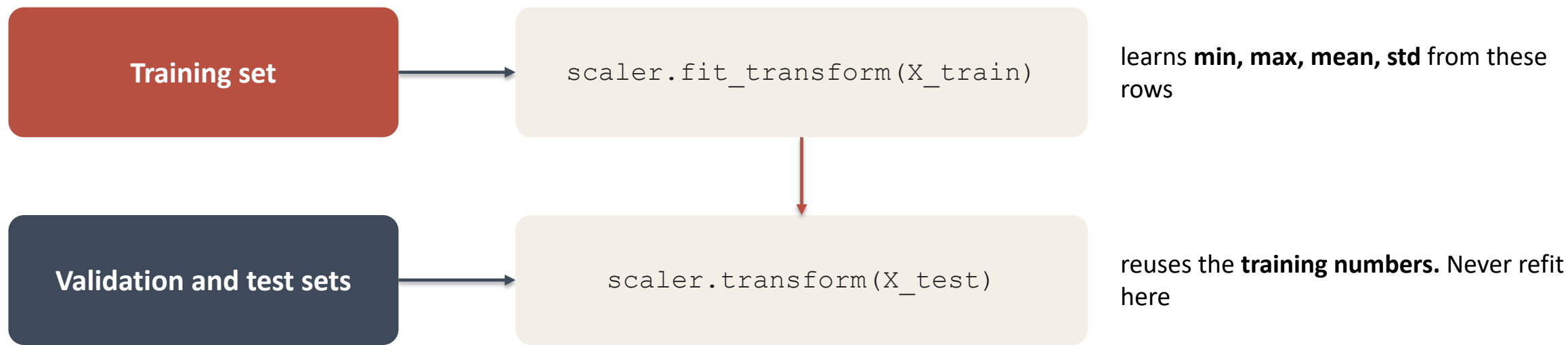
class	A	B	C	D	E	F	G
code	0	1	2	3	4	5	6

OneHotEncoder on `ocean_proximity`

row value	<1H OCEAN	INLAND	ISLAND	NEAR BAY	NEAR OCEAN
NEAR BAY	0	0	0	1	0
INLAND	0	1	0	0	0
<1H OCEAN	1	0	0	0	0

Only the ordinal code can be scaled. One-hot columns are already 0 or 1 and are left alone. Set `handle_unknown="ignore"` so a category unseen in training does not crash prediction.

Leakage: fit on training data only



Anything learned from data is part of the model: scaler statistics, imputation medians, the list of one-hot categories, PCA components.

Learn them from the training rows only, then apply to validation and test. If test rows contribute even a column mean, the held-out score is optimistic.

Watch for it in the tutorial: the scalers and the target are fitted before the split. It is a small leak, and a pipeline removes it by construction.

Train, validation, test

Training 50%

Validation 25%

Test 25%

The tutorial's split: `train_test_split` twice, half then half again. Common alternatives are 60/20/20 or 80/10/10.

Training

Fits the parameters: the weights of a network, the split points of a tree, the coefficients of a regression.

Validation

Chooses between models: how many trees, how deep, which features, when to stop training. Try many settings, keep the best.

Test

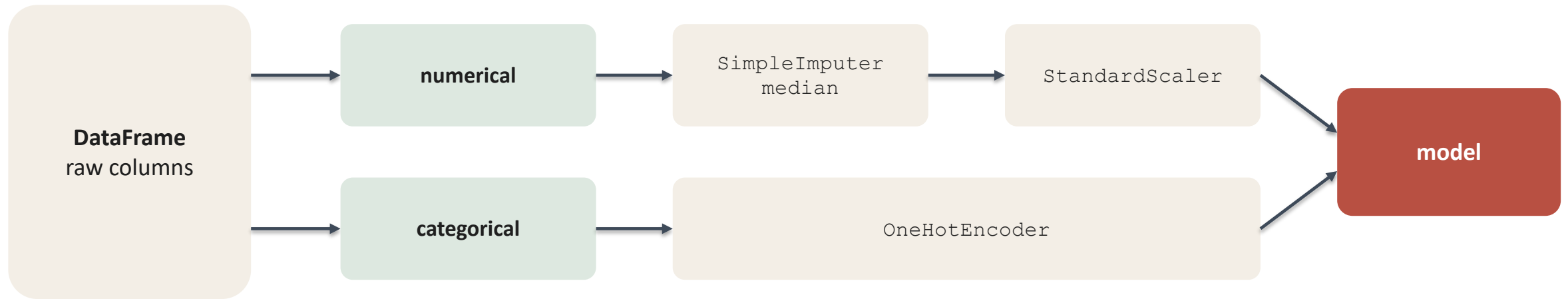
Reports the final number. Touched once, at the end, after every choice is locked in.

Why two held-out sets? Every comparison on the validation set fits to it a little. After fifty hyperparameter trials, the best validation score is partly luck. The test set is the one score no choice was based on.

Small data: cross-validation rotates the validation role across folds. The test set stays separate.

Which rows matters as much as how many: random, forward in time, or held-out stations give different answers (Week 5).

Pipelines make it repeatable



```
num_pipeline = make_pipeline(SimpleImputer(strategy="median"), StandardScaler())
cat_pipeline = make_pipeline(OneHotEncoder(handle_unknown="ignore"))
preprocessor = ColumnTransformer([("num", num_pipeline, numerical_cols),
                                  ("cat", cat_pipeline, categorical_cols)])
model = make_pipeline(preprocessor, RandomForestRegressor())
model.fit(X_train, y_train)          # learns imputer, scaler, categories, forest
model.predict(X_test)                # applies all of them, nothing refitted
```

fit learns every statistic from the training rows and predict applies them, so leakage is impossible by construction. One object to cross-validate, save, and hand to a colleague.

Before you train anything: a checklist

Explore

- ☐ Histogram of every column, and read the axes
- ☐ A map if there are coordinates, a scatter matrix for the numeric columns
- ☐ Found the caps, the sentinel codes, the categories stored as numbers
- ☐ Looked at how much is missing, and asked why
- ☐ Test set split off before any of this got too detailed

Prepare

- ☐ Missing values: codes replaced, then dropped or imputed on purpose
- ☐ Heavy tails transformed
- ☐ Numeric features scaled, ordered categories coded, unordered ones one-hot
- ☐ Every step inside a pipeline, fitted on training rows only
- ☐ Training, validation, and test sets that match how the model will be used

Then, and only then, pick a model. That is Week 3.

Checklist condensed from Géron, Hands-On Machine Learning, 3rd ed., Appendix A, and the Week 2 tutorial